

module3_lab

September 18, 2026

1 Module 3 Lab: Cleaning Messy Sales Data

Name: Mac McKay

Course: CS 82A

1.1 Overview

The main goal of this lab is to take a messy sales dataset (300 rows and 6 columns) and clean it so it's ready for analysis.

1.2 Step 1: Load and Inspect

```
[11]: import pandas as pd
```

```
df = pd.read_csv("messy_sales.csv")
print(df.shape)
print(df.dtypes)
df.head(10)
```

```
(300, 6)
order_id      int64
date          str
product       str
price         float64
qty           int64
zip           int64
dtype: object
```

```
[11]:
```

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	NaN	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405
5	1280	04/22/2026	pen set	NaN	4	30303
6	1204	04/18/2026	charger	8.41	2	2134
7	1249	2026-05-15	keyboard	75.04	4	90405
8	1037	2026-05-19	mug	9.31	2	60614

9 1177 2026-05-13 mug 10.11 3 90210

1.3 Step 2: First Observations

1. **Zip codes stored as numbers.** Pandas read `zip` as `int64`, so any zip starting with 0 lost its leading zero. Zip codes are labels, not quantities, so they should be stored as text.
2. **Dates are stored in mixed formats.** The `date` column is type object, with some dates as MM/DD/YYYY and others as YYYY-MM-DD. Can't be sorted or filtered by time until they're normalized.
3. **Some prices are missing and some quantities are negative.** There are negative values in `price`, and some rows have a negative `qty`, which doesn't make sense for a normal sale (only in returns).

1.4 Steps 3–4: Missing Values and Imputation

```
[12]: print(df.isna().sum())

median_price = df["price"].median()
print("Fill value:", median_price)
df["price"] = df["price"].fillna(median_price)

df.isna().sum()
```

```
order_id      0
date          0
product       0
price         12
qty           0
zip           0
dtype: int64
Fill value: 37.53
```

```
[12]: order_id      0
      date         0
      product      0
      price        0
      qty          0
      zip          0
      dtype: int64
```

1.5 Step 5: Remove Duplicates

```
[13]: print("Duplicates:", df.duplicated().sum())
      df = df.drop_duplicates()
      df.shape
```

```
Duplicates: 8
```

```
[13]: (292, 6)
```

1.6 Step 6: Repair Zip Codes

```
[14]: df["zip"] = df["zip"].astype(str).str.zfill(5)
print(df["zip"].str.len().value_counts())
df["zip"].head(10)
```

```
zip
5    292
Name: count, dtype: int64
```

```
[14]: 0    60614
1    30303
2    10001
3    98101
4    90405
5    30303
6    02134
7    90405
8    60614
9    90210
Name: zip, dtype: str
```

1.7 Step 7: Standardize Dates

```
[15]: df["date"] = pd.to_datetime(df["date"], format="mixed")
print("Missing dates:", df["date"].isna().sum())
df.dtypes
```

```
Missing dates: 0
```

```
[15]: order_id      int64
date      datetime64[us]
product      str
price      float64
qty      int64
zip      str
dtype: object
```

1.8 Step 8: Negative Quantities

```
[16]: df[df["qty"] < 0]
```

```
[16]:   order_id   date  product  price  qty  zip
202    1140 2026-04-18  webcam  38.69  -5  98101
262    1233 2026-05-09  desk lamp  22.64  -4  10001
297    1025 2026-04-27  keyboard  47.30  -2  02116
```

```
[17]: df["is_return"] = df["qty"] < 0
      df[df["is_return"]]
```

```
[17]:   order_id   date  product  price  qty  zip  is_return
      202     1140 2026-04-18  webcam 38.69  -5  98101      True
      262     1233 2026-05-09  desk lamp 22.64  -4  10001      True
      297     1025 2026-04-27  keyboard 47.30  -2  02116      True
```

1.9 Step 9: Save the Cleaned File

```
[18]: df.to_csv("sales_clean.csv", index=False)
```

Decision: Flag as returns.

The three negative rows have normal prices, real products, and valid dates and zips, so they look like real transactions rather than broken data.

The quantities are reasonable sizes for returns, not obvious typos. Flagging them with an `is_return` column keeps the data intact and lets later analysis include or exclude returns as needed.

With this step, the cleaning is complete and the data is ready for further analysis.

1.10 Final Preview: Before and After

Comparison of the first five rows of the original file and the cleaned dataset.

```
[19]: raw = pd.read_csv("messy_sales.csv")
      print("Before:", raw.shape)
      display(raw.head())

      print("After:", df.shape)
      display(df.head())

      print("Zips with restored leading zeros:")
      display(df[df["zip"].str.startswith("0")].head(3))
```

Before: (300, 6)

	order_id	date	product	price	qty	zip
0	1254	04/06/2026	mug	11.36	4	60614
1	1057	2026-05-30	charger	14.79	3	30303
2	1150	05/06/2026	mug	5.50	1	10001
3	1066	05/30/2026	notebook	NaN	1	98101
4	1114	04/06/2026	webcam	45.59	4	90405

After: (292, 7)

	order_id	date	product	price	qty	zip	is_return
0	1254	2026-04-06	mug	11.36	4	60614	False
1	1057	2026-05-30	charger	14.79	3	30303	False
2	1150	2026-05-06	mug	5.50	1	10001	False

3	1066	2026-05-30	notebook	37.53	1	98101	False
4	1114	2026-04-06	webcam	45.59	4	90405	False

Zips with restored leading zeros:

	order_id	date	product	price	qty	zip	is_return
6	1204	2026-04-18	charger	8.41	2	02134	False
13	1108	2026-05-16	desk lamp	21.63	5	02116	False
21	1090	2026-05-01	keyboard	53.07	1	02116	False

Repo: <https://github.com/raulmckay-blip/cs82a-portfolio>